CampusLens — Spécification du projet

Nov 4, 2026 · @Ousmane Barry

1. Présentation du projet

CampusLens est une plateforme web gratuite qui aide les étudiants en France à **connaître les aides auxquelles ils ont droit** et à **évaluer leur budget** pour mesurer leur niveau de précarité.

Contexte. À l'été 2026, l'État a restreint les exonérations de frais d'inscription pour les étudiants hors Union européenne. Le projet est parti de leur situation. Il s'est vite élargi : loyer, alimentation, frais de rentrée touchent tous les étudiants ; les étudiants internationaux en souffrent simplement plus.

Objectif. Donner à chaque étudiant, en quelques minutes et sans compte :

- son reste-à-vivre mensuel et un niveau de situation (équilibrée, fragile, en difficulté) ;
- la liste des aides adaptées à son profil, avec la démarche pour les obtenir ;
- les repères pour préparer sa rentrée et choisir sa ville.

Public visé.

Profil	Besoins spécifiques
Étudiant français ou européen	Bourse, aide au logement, aides d'urgence
Étudiant boursier	Exonérations, logement CROUS prioritaire, tarifs sociaux
Étudiant international hors UE (visa étudiant)	Droits d'inscription différenciés, exonération, limite de 964 h de travail par an, accès restreint à certaines aides

Principe directeur. Les données ne sont pas saisies une fois pour toutes : elles viennent des sources officielles et se mettent à jour automatiquement (section 3).

2. Fonctionnalités, page par page

Le site compte six pages, reliées par une barre de navigation latérale. Aucune page ne demande de compte.

Page	Rôle	Fonctionnalités	Données utilisées
Accueil	Orienter l'étudiant	Bandeau « Comprendre pour agir ensemble », « Comment ça marche » en 3 étapes, aides à connaître, raccourcis par profil, « Bon à savoir »	Aides mises en avant
Évaluer ma situation	Mesurer la précarité	Profil (statut, niveau, boursier, exonéré), revenus et dépenses, reste-à-vivre en direct, jauge, niveau de situation, recommandations, bilan PDF	Frais d'inscription, CVEC, seuils, aides
Mes aides	Faire connaître les droits	Filtre par profil et par besoin, badge d'éligibilité par aide, démarche dépliable, onglets Associations et Comparateur de villes	Aides, éligibilités, associations, villes
Préparer mon budget	Anticiper la rentrée	Indicateurs par profil, frais de rentrée poste par poste avec l'aide qui l'allège, tableau des règles par profil	Postes de rentrée, frais, règles
Ressources	Donner accès aux sources	Recherche, filtre par catégorie, liste des démarches et documents officiels à jour	Sources, documents
À propos	Présenter le projet	Principes clés, mission, liens et contact	Contenu fixe

Règles de calcul du simulateur.

- Reste-à-vivre = total des revenus − total des dépenses − (droits d'inscription + CVEC) ÷ 12.
- Droits d'inscription : 0 € si boursier ; tarif différencié si international hors UE non exonéré ; sinon tarif national. Ces montants sont lus en base, jamais écrits dans le code.
- Niveau : « en difficulté » sous 0 €, « fragile » sous le seuil (150 € par défaut, réglable en base), « équilibré » au-dessus.
- Les recommandations sont choisies selon le profil, le résultat et la part du loyer dans les revenus.

- Le calcul se fait dans le navigateur ; aucune simulation n'est envoyée ni enregistrée.

3. Des données à la source, toujours à jour

Aucun chiffre n'est écrit en dur dans le site : chaque montant, règle, aide ou document vient d'une source officielle, et la plateforme va le chercher elle-même à intervalles réguliers. Quand la source change, la nouvelle valeur est utilisée dès la simulation suivante.

Trois niveaux d'automatisation, selon ce que la source permet :

Niveau	Quand la source…	Ce que fait CampusLens	Exemples
1. Synchronisation automatique	publie une API ou un jeu de données ouvert	Un robot télécharge les données, les compare, enregistre la nouvelle version et l'utilise aussitôt	data.gouv.fr, data.enseignementsup-recherche.gouv.fr
2. Calcul à la source	publie ses règles sous forme de moteur de calcul	Le simulateur interroge ce moteur au lieu de recopier les règles	Moteur OpenFisca pour les aides sociales
3. Veille avec validation	publie une page web ou un PDF sans API	Un robot détecte que la page ou le document a changé, alerte l'administrateur, qui valide la nouvelle valeur en un clic	Indicateur FAGE, pages CROUS, décrets

Garanties pour l'utilisateur.

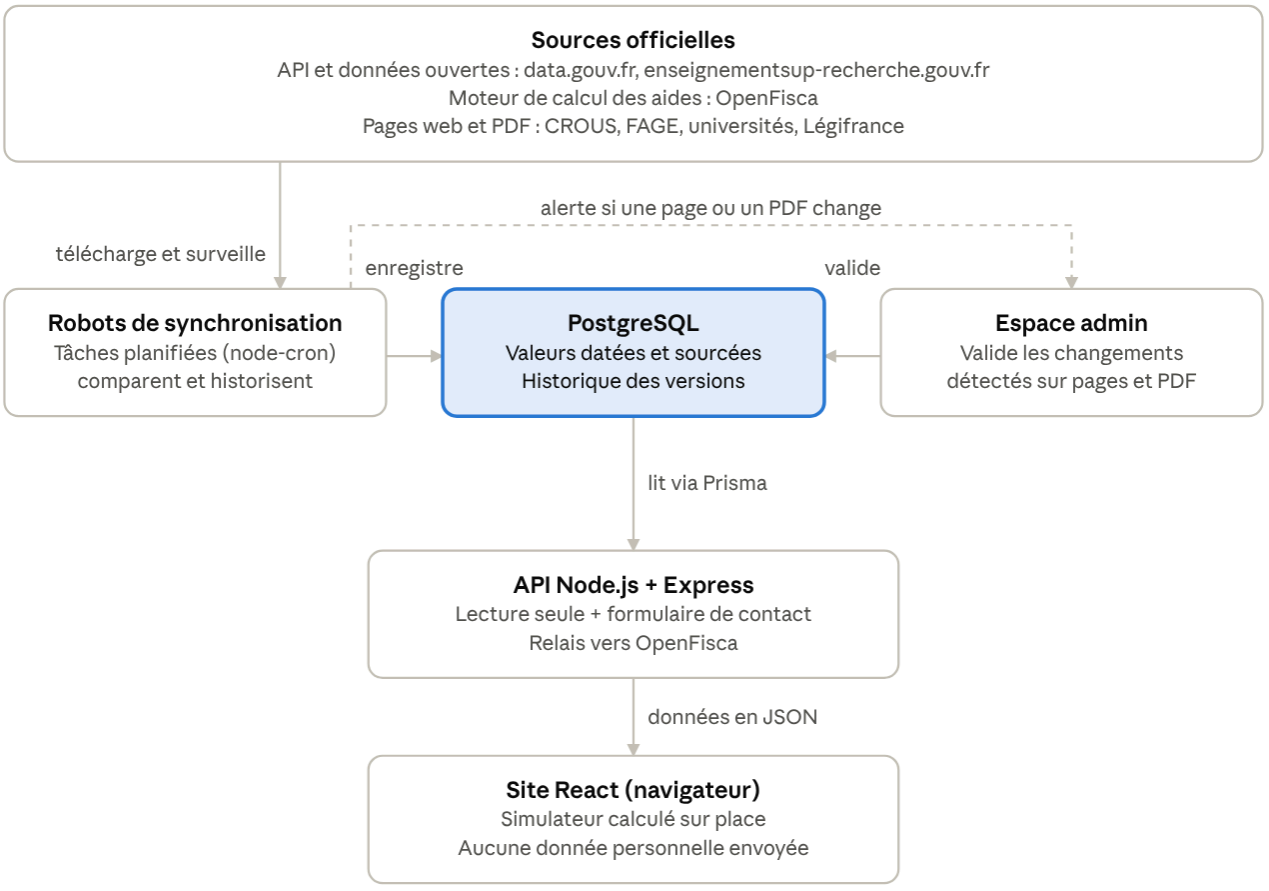
- Chaque valeur affiche sa source et sa date de dernière vérification.
- Une valeur qui n'a pas été vérifiée depuis trop longtemps (délai propre à chaque source) est signalée « à vérifier » sur le site.
- Chaque changement est historisé : on garde l'ancienne valeur, la nouvelle, la date et la source. Une erreur de source peut donc être annulée.
- Les documents de la page Ressources pointent vers la version officielle en ligne ; un lien cassé ou un document remplacé est détecté et signalé.

- Les montants qui dépendent de l'année (droits d'inscription, CVEC) sont stockés par année universitaire : le simulateur prend toujours ceux de l'année en cours.

4. Architecture technique

Tout le projet est écrit en JavaScript : React pour le site, Node.js pour le serveur et les robots, PostgreSQL pour la base. Le serveur n'expose que de la lecture, sauf le formulaire de contact.

Les données vont des sources officielles au simulateur, sans saisie à la main



architecture de CampusLens · 6 briques

Les robots alimentent la base sans intervention ; l'administrateur n'intervient que pour valider ce qui vient de pages web ou de PDF.

Brique	Technologie	Rôle
Site	React + Vite, HTML, CSS, React Router	Les six pages, le simulateur, le bilan PDF (généré dans le navigateur)

Brique	Technologie	Rôle
API	Node.js (version LTS) + Express	Sert les données en JSON, reçoit le formulaire de contact, relaie les calculs OpenFisca
Accès à la base	Prisma	Décrit les tables en code, protège des injections SQL, gère les migrations
Base de données	PostgreSQL	Stocke les valeurs, leur source, leur date et leur historique
Robots	node-cron (ou GitHub Actions)	Lancent les synchronisations selon la fréquence de chaque source
Contrôle des données	Zod	Vérifie le format des données reçues avant tout enregistrement
Hébergement	Vercel (site), Render ou Railway (API + robots), Neon ou Supabase (PostgreSQL)	Offres gratuites suffisantes pour démarrer

Organisation du code.

```
campuslens/
├── frontend/              # React + Vite
|   ├── src/pages/          # Accueil, Simulateur, Aides, Budget, Ressources,
APropos
|   ├── src/components/     # Sidebar, Jauge, CarteAide, BadgeEligibilite…
|   └── src/lib/simulation.js   # calcul du reste-à-vivre
├── backend/               # Node.js + Express
|   ├── prisma/schema.prisma
|   ├── src/routes/         # /aides, /frais, /villes, /sources, /contact
|   ├── src/sync/           # un fichier par source (dataGouv.js,
openfisca.js, veille.js…)
|   └── src/admin/          # validation des changements détectés
└── .env                   # secrets, jamais publié
```

5. Base de données

La base compte 16 tables en trois familles : les **données de référence** affichées sur le site, la **traçabilité** qui les garde à jour, et l'**administration**. Toute donnée affichée est rattachée à une source et à une date de vérification.

5.1 Données de référence

source — chaque origine de données (site, API, document).

Champ	Type	Description
id	entier, clé	Identifiant
nom	texte	Ex. « Carte des loyers 2025 »
organisme	texte	Ex. ANIL, CNOUS, DILA
url	texte	Adresse surveillée ou interrogée
type	liste : api, jeu_ouvert, moteur_calcul, page_web, pdf	Détermine le niveau d'automatisation
frequence_heures	entier	Intervalle entre deux vérifications
empreinte	texte	Signature (hash) du dernier contenu lu, pour détecter un changement
dernier_controle	date-heure	Dernière vérification
dernier_changement	date-heure	Dernière fois que le contenu a changé
statut	liste : ok, a_verifier, erreur	« a_verifier » si le délai de vérification est dépassé
licence	texte	Ex. Licence Ouverte 2.0, mention à afficher

annee_universitaire — id, libelle (« 2026-2027 »), date_debut, date_fin, en_cours (booléen). Une seule année est « en cours » ; le simulateur l'utilise par défaut.

profil — id, code (`fr_ue` , `boursier` , `intl`), libelle.

parametre — tous les montants et seuils utilisés dans les calculs. Aucun chiffre n'est écrit dans le code.

Champ	Type	Description
id	entier, clé	Identifiant
cle	texte	Ex. `droits_inscription` , `cvec` , `seuil_fragile` , `heures_travail_max`

Champ	Type	Description
niveau	liste : licence, master, aucun	Si la valeur dépend du niveau
profil_id	lien → profil, facultatif	Si la valeur dépend du profil
annee_id	lien → annee_universitaire	Année de validité
valeur	décimal	Ex. 178, 2 902, 105
unite	texte	€, € / an, heures / an
source_id	lien → source	Origine
verifie_le	date-heure	Dernière vérification

aide — id, slug, nom, organisme, description, categorie (logement, etudes, urgence, alimentation), frequence (mensuelle, ponctuelle…), comment_demander, lien_demarche, code_openfisca (si l'aide est calculable par OpenFisca), source_id, verifie_le, active.

eligibilite — aide_id + profil_id (clé composée), statut (oui, sous_conditions, non), precision (texte court affiché dans le badge). C'est elle qui produit « Vous y avez droit » sur la page Mes aides.

poste_rentree — id, annee_id, nom (dépôt de garantie, matériel…), montant, astuce, aide_id (facultatif : l'aide qui allège ce poste), source_id, verifie_le.

ville — id, code_insee, nom, region, loyer_m2_appartement (€/m², carte des loyers), loyer_studio_estime (calculé : loyer au m² × surface type), nb_logements_crous, abonnement_transport, source_id, verifie_le.

logement_crous — id, ville_id, nom_residence, adresse, type_logement, loyer_min, loyer_max, source_id, verifie_le.

association — id, nom, description, lien, ville_id (vide si nationale), source_id, verifie_le.

ressource — documents et démarches de la page Ressources : id, titre, type (démarche, rapport, texte officiel…), categorie, organisme, annee, url, empreinte, statut_lien (ok, deplace, casse), source_id, verifie_le.

5.2 Traçabilité et mise à jour

synchronisation — un passage de robot : id, source_id, debut, fin, statut (succes, aucun_changement, en_attente_validation, echec), nb_changements, message_erreur.

historique_valeur — chaque modification de donnée : id, table_cible, ligne_id, champ, ancienne_valeur, nouvelle_valeur, synchronisation_id, change_le. Permet d'afficher « mis à jour le … » et d'annuler une erreur.

changement_detecte — modifications trouvées sur une page web ou un PDF (niveau 3) : id, source_id, synchronisation_id, resume, ancienne_empreinte, nouvelle_empreinte, statut (a_valider, valide, rejete), traite_par (lien → administrateur), traite_le.

5.3 Administration

administrateur — id, email, mot_de_passe_hash (bcrypt), role (admin, relecteur), derniere_connexion.

message_contact — id, objet, message, email, cree_le, supprimer_apres (date de suppression automatique, RGPD).

5.4 Relations principales

- Une **source** alimente plusieurs paramètres, aides, villes, ressources ; chaque ligne affichée pointe vers sa source.
- Une **aide** a une ligne d'**eligibilite** par **profil** (relation plusieurs-à-plusieurs).
- Un **parametre** et un **poste_rentree** appartiennent à une **annee_universitaire**.
- Une **ville** a plusieurs **logement_crous** et **association**.
- Une **synchronisation** produit des lignes d'**historique_valeur** et, pour les pages web et PDF, des **changement_detecte**.

Aucune table ne stocke de simulation : le profil et le budget saisis par l'étudiant restent dans son navigateur.

6. Sources et synchronisation

Les loyers, les logements CROUS, les fiches d'aides et les calculs d'aides peuvent être récupérés automatiquement ; les frais d'inscription et le coût de la rentrée demandent une veille avec validation, faute d'API.

Donnée sur le site	Source	Format	Niveau	Vérification
Loyers par ville	Carte des loyers (ministère, estimations ANIL)	CSV par commune, annuel	1 · automatique	Chaque semaine (détection de la nouvelle édition)
Logements CROUS	Logements proposés par les CROUS (CNOUS)	Jeu de données ouvert	1 · automatique	Chaque semaine ; fraîcheur à surveiller (mise à jour irrégulière)

Donnée sur le site	Source	Format	Niveau	Vérification
Restaurants CROUS	Lieux de restauration des CROUS (CNOUS)	Jeu de données ouvert	1 · automatique	Chaque semaine
Fiches d'aides et démarches	Fiches pratiques Service-Public (DILA)	XML, mis à jour chaque jour	1 · automatique	Chaque jour
Montant estimé des aides (APL…)	API OpenFisca (ANCT)	API ouverte	2 · calcul à la source	À chaque simulation
Droits d'inscription, CVEC	Arrêtés annuels (Légifrance), pages des universités	Texte officiel, page web	3 · veille + validation	Chaque jour de mai à septembre, puis chaque mois
Coût de la rentrée	Indicateur FAGE	PDF annuel	3 · veille + validation	Chaque semaine en août et septembre
Documents de la page Ressources	Sites des organismes (OVE, SIES, Campus France…)	Pages web, PDF	3 · veille des liens	Chaque semaine

Piste à étudier : le simulateur public aides-jeunes de l'État est open source (licence AGPL-3.0) ; son catalogue d'aides pourrait compléter la table `aide`, à condition de respecter sa licence.

Déroulé d'une synchronisation (un robot par source, dans `backend/src/sync/`) :

1. Télécharger les données ou la page.
2. Vérifier le format avec Zod ; en cas d'erreur, s'arrêter sans rien modifier et marquer la source « erreur ».
3. Calculer l'empreinte du contenu et la comparer à `source.empreinte` ; si rien n'a changé, mettre à jour `dernier_controle` et s'arrêter.
4. Niveaux 1 et 2 : enregistrer les nouvelles valeurs et écrire chaque différence dans `historique_valeur`.
5. Niveau 3 : créer un `changement_detecte` « à valider » et envoyer une alerte à l'administrateur ; la valeur n'est modifiée qu'après sa validation.
6. Enregistrer le passage dans `synchronisation` (statut, nombre de changements).

Toutes les données réutilisées sont sous Licence Ouverte : le site doit citer chaque producteur, comme la licence l'exige.

7. Sécurité, RGPD et réalisation

Sécurité.

- L'API publique est en lecture seule ; seul `POST /contact` écrit, avec une limite d'envois par minute (express-rate-limit) et une vérification anti-robot.
- Toutes les requêtes passent par Prisma : pas de SQL écrit à la main, donc pas d'injection SQL.
- Les données reçues des sources et du formulaire sont validées avec Zod avant enregistrement.
- L'espace admin est protégé par mot de passe haché (bcrypt), session sécurisée et en-têtes HTTP de sécurité (Helmet).
- Les secrets (connexion à la base, clés) restent dans `.env`, jamais sur GitHub.

RGPD.

- Pas de compte, pas de cookie de suivi : le simulateur ne transmet rien.
- Seul le formulaire de contact collecte un e-mail ; il est supprimé automatiquement à la date `supprimer_apres` (6 mois proposés).
- Une page « Mentions légales et confidentialité » indique le responsable du site et la durée de conservation.

Limites à afficher. Les résultats sont indicatifs et ne remplacent ni le CROUS, ni la CAF, ni un service social. Une source sans API peut avoir quelques jours de décalage avant validation.

Étapes de réalisation.

1. Maquette validée (fait).
2. Base de données : `schema.prisma`, migrations, remplissage initial des profils, paramètres 2026-2027 et aides.
3. API Express : routes de lecture et formulaire de contact.
4. Site React : les six pages branchées sur l'API, simulateur et bilan PDF.
5. Robots de niveau 1 : carte des loyers, CROUS, fiches Service-Public.
6. Robot de niveau 3 et espace admin de validation.
7. Branchement OpenFisca pour estimer les aides au logement.
8. Tests, mise en ligne, présentation.

Sources

- Carte des loyers 2025 — data.gouv.fr
- Logements proposés par les CROUS — data.gouv.fr
- Lieux de restauration des CROUS — data.gouv.fr
- Fiches pratiques Service-Public — data.gouv.fr
- API OpenFisca — data.gouv.fr
- aides-jeunes — code.gouv.fr
- Coût de la rentrée 2026 selon la FAGE — L'Étudiant
- Droits d'inscription différenciés 2026 — service-public.fr